Multi-Cluster Scheduling of DAG-Based Tasks

Using Priority Functions

Marisha Thorat

Supervisor: Prof. Preeti Malakar

April 24, 2026

Motivation

HPC Workloads

- Applications consist of **dependent tasks (DAGs)**
- Executed across **CPU and GPU clusters**

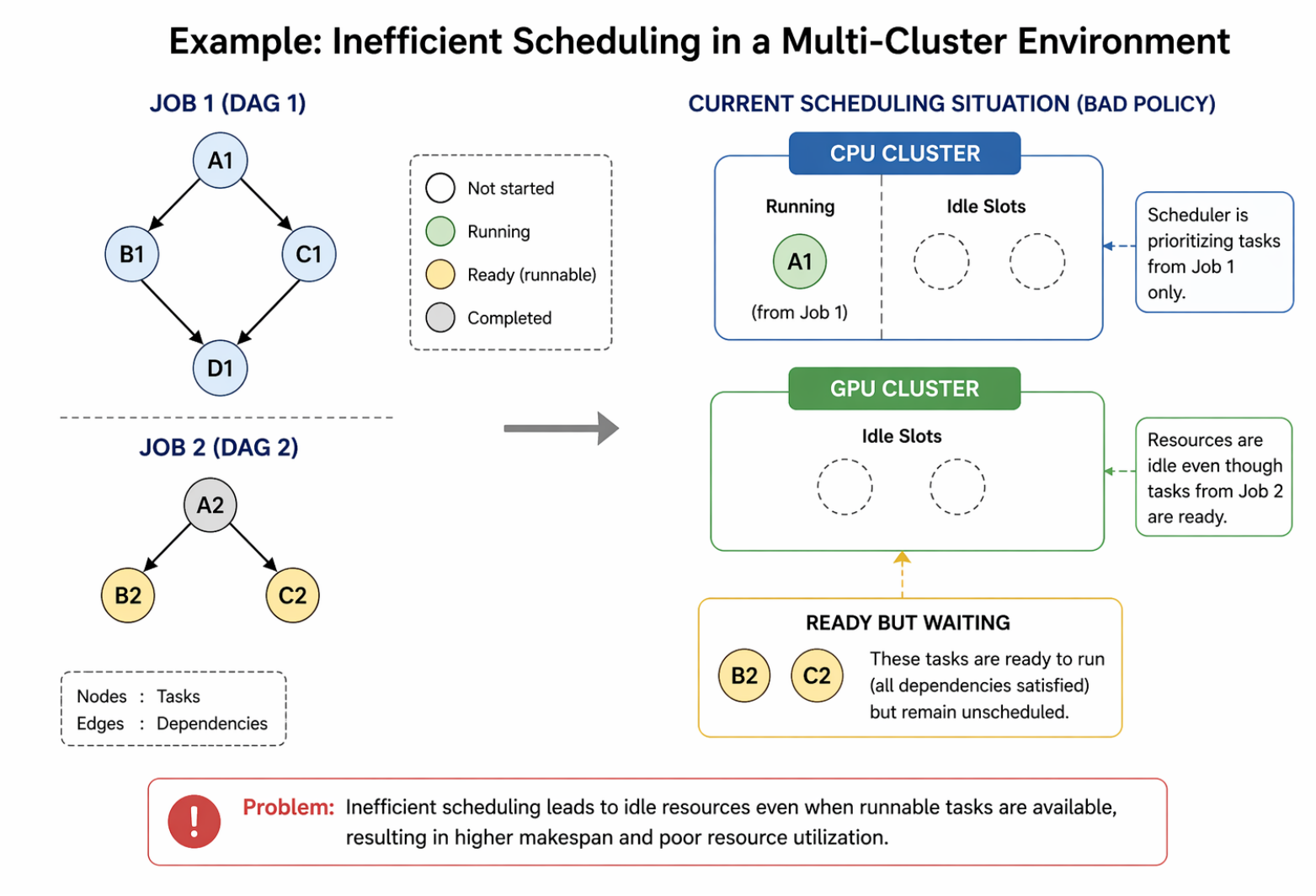
Inefficient Scheduling

- Ignores task dependencies
- Leads to **idle resources despite runnable tasks**
- Increases **makespan**

Limitation of Existing Approaches

- FCFS and similar policies lack:
  - Critical-path awareness
  - Dependency-driven prioritization

Inefficient Scheduling Example



Runnable tasks remain unscheduled, leading to idle resources

Problem Statement

Goal

Design a **multi-cluster scheduling algorithm** for DAG-based tasks across **CPU and GPU resources**

Key Challenges

- Respect **task dependencies**
- Minimize **makespan**
- Balance **resource utilization and fairness**
- Handle **heterogeneous workloads**

Which task should be scheduled next?

Related Work: FCFS with Backfilling

How It Works

- Tasks scheduled in order of **ready time (FCFS)**
- Head task gets a **reservation** if resources unavailable
- Other tasks are **backfilled** without delaying the head

Limitations

- No **critical-path awareness**
- No **global optimization metric**
- Suboptimal for **dependent and heterogeneous workloads**

Proposed Methodology

- Represent tasks as nodes in a **DAG**
- Compute **critical path (CP)** for each task
- Identify **runnable tasks** (dependencies satisfied)
- Compute **priority score** for each task
- Select highest-priority task under **resource constraints**
- Update system state and repeat

$$\text{DAG} \rightarrow \text{CP} \rightarrow \text{Runnable Tasks} \rightarrow \text{Priority} \rightarrow \text{Schedule}$$

Priority Function

$$\textbf{Priority} = w_1 \cdot CP + w_2 \cdot fit - w_3 \cdot remWork + \alpha \cdot waiting - w_4 \cdot depPenalty$$

- **CP**: Critical path length
- **fit**: Resource utilization
- **remWork**: Remaining work
- **waiting**: Waiting time
- **depPenalty**: Dependency penalty

Select task with highest priority

Scheduling Logic

- Identify **runnable tasks**
- Compute **priority** for each task
- Select best task under **resource constraints**
- Execute task and update dependencies

```
while(tasks remaining) {
    select best task;
        execute;
      update; }
```

Implementation

- Custom scheduler implemented in **C++**
- Models a **multi-cluster system** (CPU and GPU resources)
- Tasks represented as **DAGs with dependencies**
- Dynamic scheduling using **priority-based selection**
- Event-driven execution:
  - Task arrival
  - Task completion
- Explored **SLURM** for real-world scheduling insights

Experimental Setup

Workload Design

- Tasks organized as **DAGs**
- Multiple jobs with **different structures**

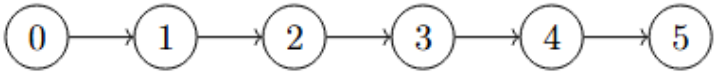
Variations Considered

- DAG types: **Linear, Parallel, Fork-Join, Diamond, Mixed**
- Arrival patterns: **simultaneous, staggered, bursty**
- Task heterogeneity: **heavy vs light tasks**
- Resource types: **CPU and GPU clusters**

Evaluation

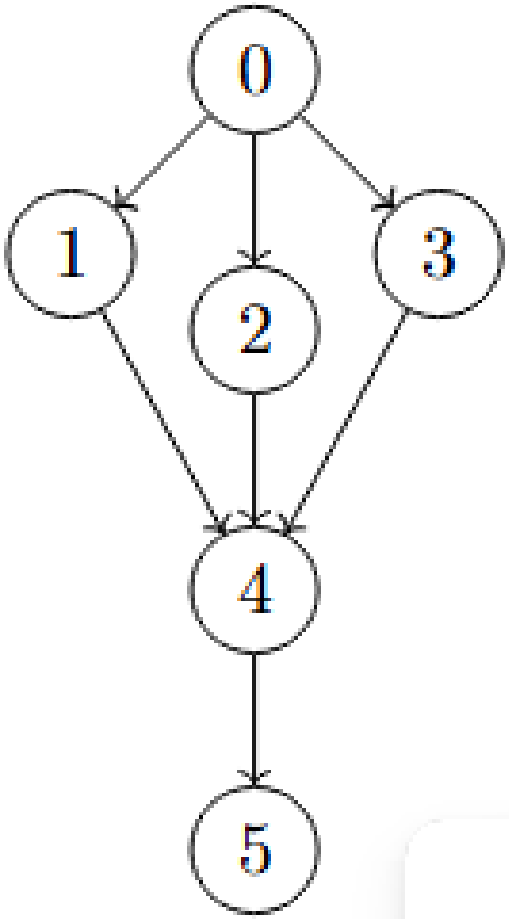
- Compared with **FCFS + Backfilling**
- Metric: **Makespan**
- **10 diverse scenarios (TC1–TC10)**

DAG Structures Used in Experiments

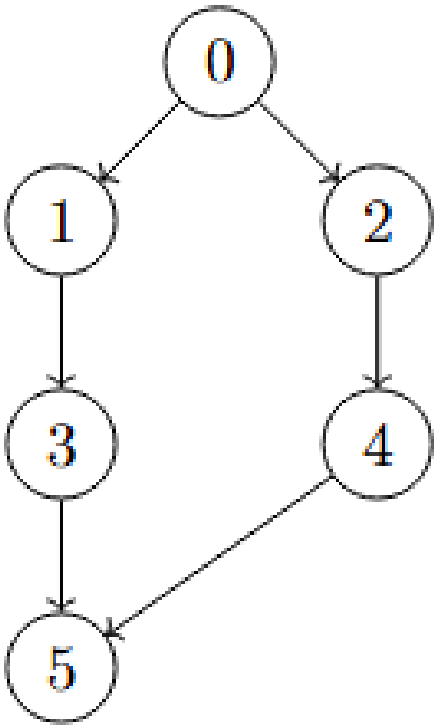


Linear

Parallel



Fork-Join



Diamond

Weight Configuration

Two configurations were evaluated

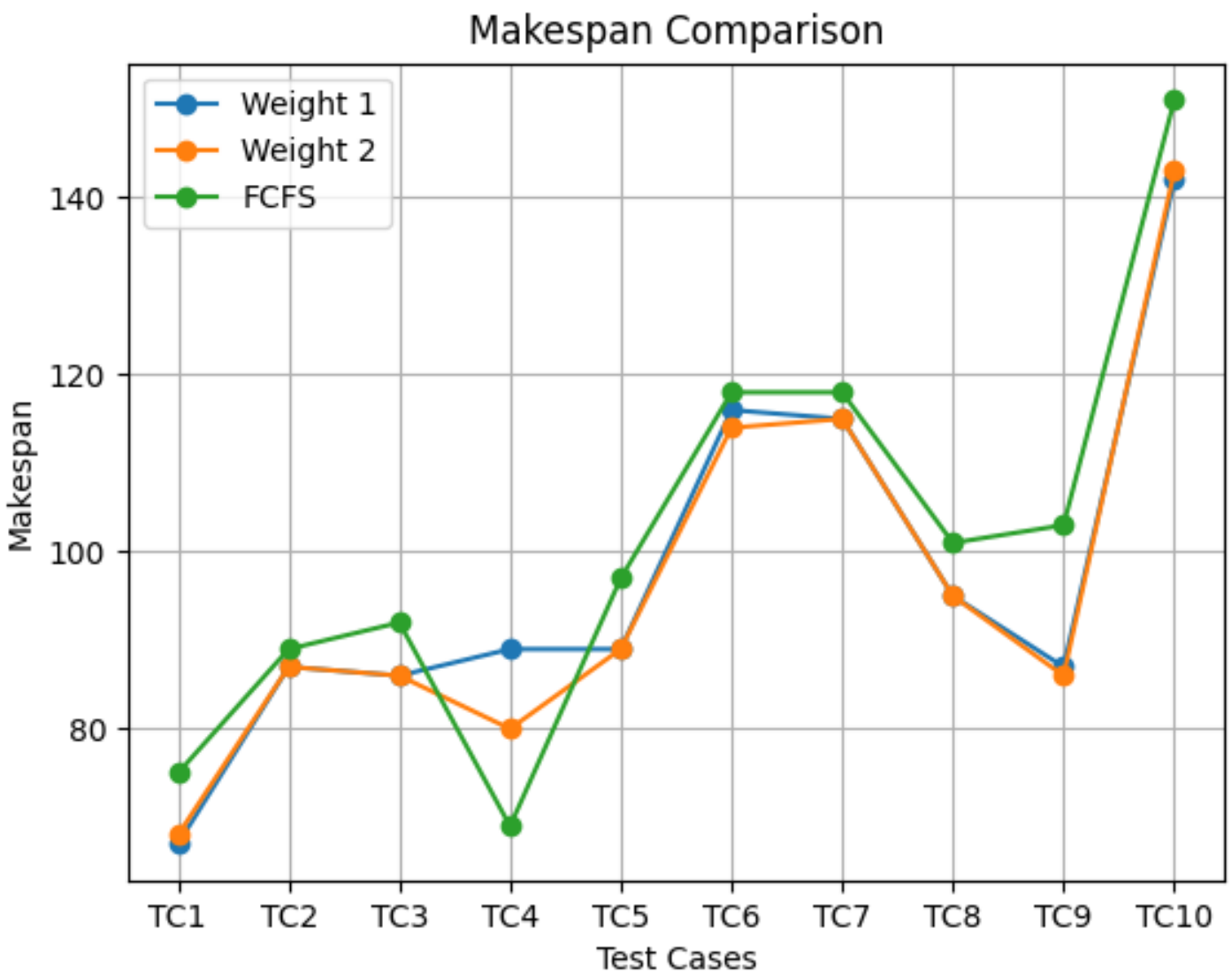
Configuration 1: Critical-Path Oriented

- Higher weight on **critical path (CP)**
- Focus on reducing **makespan**

Configuration 2: Fairness-Oriented

- Higher weight on **waiting time and remaining work**
- Focus on **fairness and responsiveness**

Results: Makespan Comparison



Proposed method outperforms FCFS in most test cases

Results: Key Insights

- Average improvement: **7–8%**
- Peak improvement: **15%**

Test Case	W1 Improvement (%)	W2 Improvement (%)
TC1	10.67	9.33
TC2	2.25	2.25
TC3	6.52	6.52
TC4	-28.99	-15.94
TC5	8.25	8.25
TC6	1.69	3.39
TC7	2.54	2.54
TC8	5.94	5.94
TC9	15.53	16.50
TC10	5.96	5.30

TC9 shows highest gain, while TC4 highlights limitations in synchronization-heavy cases

Performance Analysis

Strong Performance

- Dynamic and bursty workloads
- Heterogeneous tasks (CPU + GPU)
- Better resource utilization

Limited Improvement

- Simple DAGs (low scheduling flexibility)

Performance Drop

- Synchronization-heavy DAGs (Diamond)
- Parallel branches must complete before merging
- Limits scheduling flexibility

Weight Configuration Analysis

Configuration 1: Critical-Path Focused

- Prioritizes long dependency chains
- Reduces makespan

Configuration 2: Fairness-Oriented

- Prioritizes waiting tasks
- Improves responsiveness

Observation

- Fairness-based configuration performs better in dynamic workloads
- Highlights trade-off between:
  - Throughput
  - Fairness

Conclusion

- Proposed a **priority-based scheduler** for DAG-based tasks
- Improves **resource utilization** and reduces **makespan**
- Achieves **7–8% average improvement** (up to **15%**)
- Performance depends on **workload characteristics**
- Demonstrates trade-off between:
  - Critical-path optimization
  - Fairness

Future Work

- Adaptive weight tuning
- Integration with real schedulers (SLURM)
- Larger-scale evaluation